Problem 1.

Process:

86 images were taken of a 11 x 8 checkerboard pattern with square sizes of 23.38 mm with an Arducam IMX296 GS camera with a telephoto lens sold by Raspberry Pi. The images produced had a resolution of 1456 x 1088. The checkerboard pattern moved around the image frame, tilted up to 30 degrees in all dimensions, and moved toward and away from the camera. Special attention was given to the periphery of the frame, as that is especially important for distortion calculation.

To calibrate the camera, ChatGPT was prompted with:

> "Write me a program using the OpenCV library in python that will read all the images from a folder and use them to get calibration parameters for the camera. Use lots of comments in the code so it is understandable to someone without a coding background. Use a placeholder file location for a windows 11 machine, I will provide the file locations when I run it myself."

The checkerboard square and pattern size was input into the "1_camera_calibration.py", and then it was run with the 86 collected images. The program failed to calibrate six images. These six, and two additional images with a pixel error of over one were removed from the data set. The data set was then reduced in number to 25, 15, 10, 5, and 2 images to compare data set size verses quality of the calibration.

To validate the calibration, "1_camera_validation.py" was generated by ChatGPT and a test image that was not part of the original dataset was assessed with the calibration matrix of each image set.

Results:

The calibration matrixes were evaluated based on their reprojection error and their error when run on the validation test. As seen in Figure 1, there is a significant gain in accuracy when increasing the sample set of a calibration that diminishes rapidly as the sample size increases.

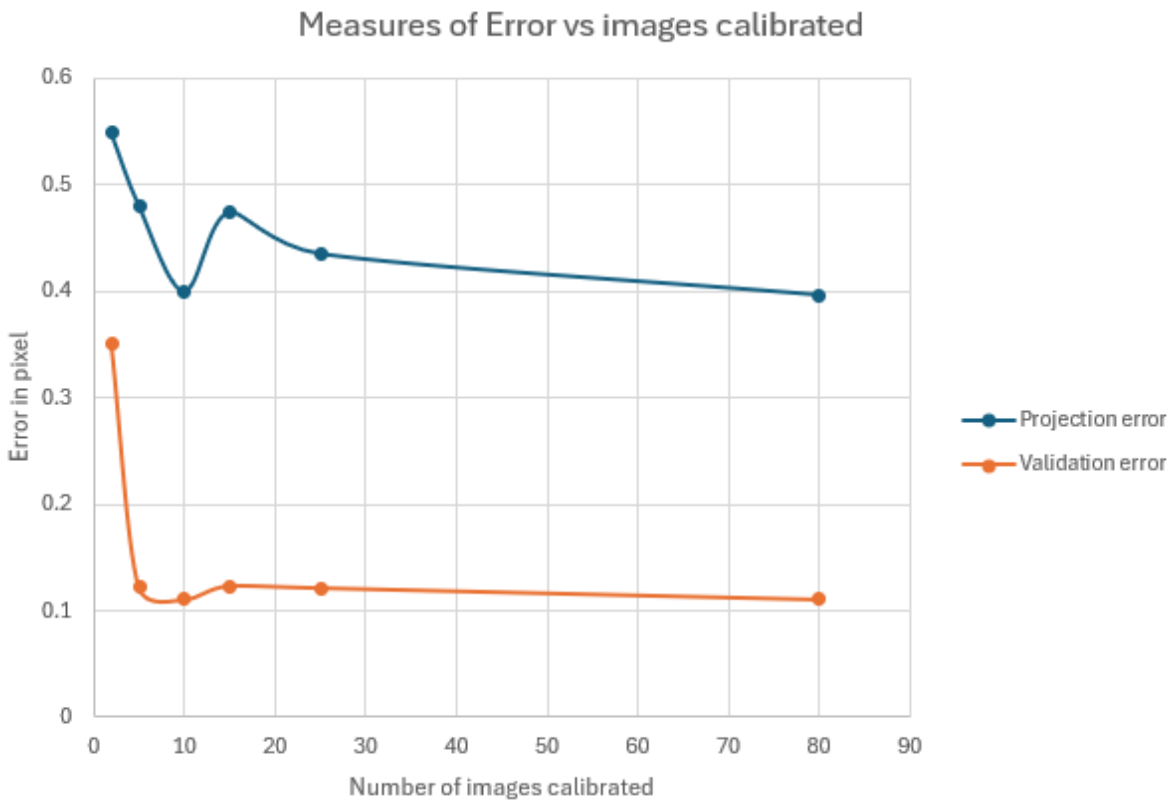


Figure 1 - Measures of Error vs images calibrated

Problem 2-3.

Process:

Code, "2_3_noise filtering.py" was generated with ChatGPT to run a provided selection of images through a gaussian kernel, median, bilaterial, and n-rank filters using the following prompt:

> "Write me a program using the OpenCV library in python that will read all the images from a folder and apply gaussian kernel, median filter, bilateral, and n-rank filter in both 3x3 and 5x5 windows. Save an append them into a single image block with labels for each original file. Use lots of comments in the code so it is understandable to someone without a coding background. Use a placeholder file location for a windows 11 machine, I will provide the file locations when I run it myself. I am working in a VScode ide with a python file."

Because the results of n-rank filtering are dependent on the value selected to filter by, high, medium, and low values were selected for each window size.

Results:

All filtering permutations are shown below for each image. In the “UniformNoise.jpg”, the gaussian, median, and bilateral filters all produce images with reduced noise. The bilaterial filter in 3x3 produces the best noise reduction while keeping a crisp image. Each 5x5 window produces noticeably more blurring of edges without any benefit in image quality. The high and low n-rank filters significantly reduce the quality of the image.

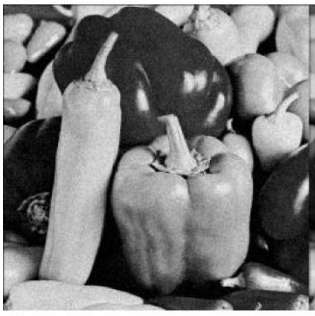
In “GaussianNoise.jpg”, the 5x5 gaussian filter is a clear winner for cleaning the noise. It reduces uniformly distributed grain of the original picture and softens the edges, restoring the organic shapes and homogenizing each object.

In “camera_man_w_noise.jpg”, white speckled dots are present throughout the image. The median 3x3 filter does the best job of removing those white spots without destroying the edge detail of the image like the gaussian filter does. The bilateral filter does a good job on the background but fails to eliminate the noise from the subject.

Unsurprisingly, the median filter and middle rank n-rank filters do the best job removing the noise from “SaltAndPepperNoise.jpg.” The 3x3 filter removes every trace of the noise while retaining detail. The 5x5 12-rank, 5x5 median, and 3x3 4-rank produce a similar effect, but either fail to completely remove the noise or blur edges unnecessarily.

Original file: UniformNoise.jpg

Original



Gaussian - 3x3

Gaussian - 5x5

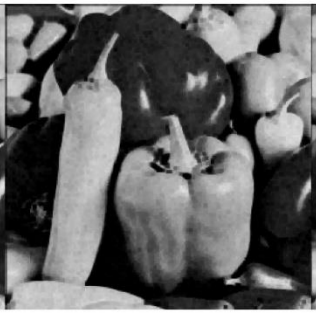
Median - 3x3

Median - 5x5

Bilateral - 3x3

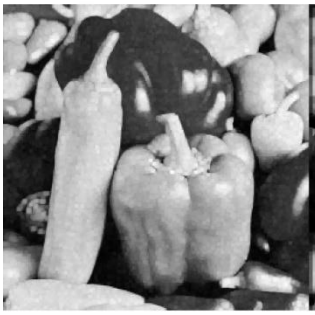
Bilateral - 5x5

N-Rank 1 - 3x3

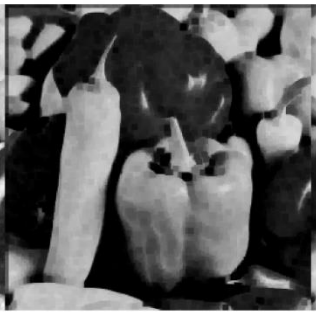


N-Rank 4 - 3x3

N-Rank 9 - 3x3



N-Rank 1 - 5x5

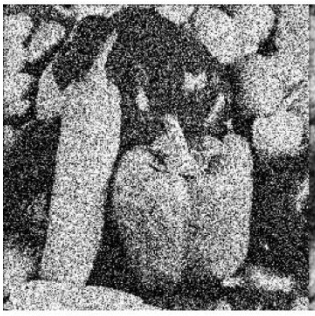


N-Rank 12 - 5x5

N-Rank 25 - 5x5

Original file: GaussianNoise.jpg

Original



Gaussian - 3x3

Gaussian - 5x5

Median - 3x3

Median - 5x5

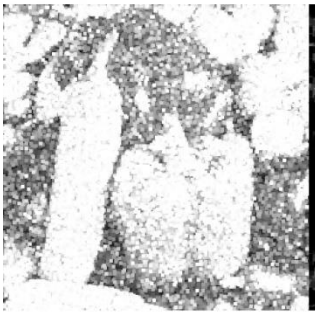
Bilateral - 3x3

Bilateral - 5x5

N-Rank 1 - 3x3

N-Rank 4 - 3x3

N-Rank 9 - 3x3



N-Rank 1 - 5x5

N-Rank 12 - 5x5

N-Rank 25 - 5x5

Original file: camera_man_w_noise.jpg

Original

Gaussian - 3x3

Gaussian - 5x5

Median - 3x3

Median - 5x5

Bilateral - 3x3

Bilateral - 5x5

N-Rank 1 - 3x3

N-Rank 4 - 3x3

N-Rank 9 - 3x3

N-Rank 1 - 5x5

N-Rank 12 - 5x5

N-Rank 25 - 5x5

Original file: SaltAndPepperNoise.jpg

Original

Gaussian - 3x3

Gaussian - 5x5

Median - 3x3

Median - 5x5

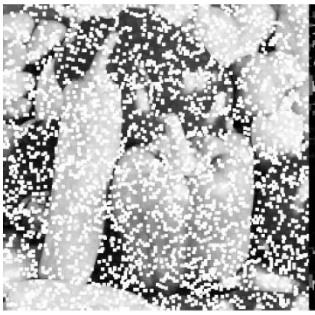
Bilateral - 3x3

Bilateral - 5x5

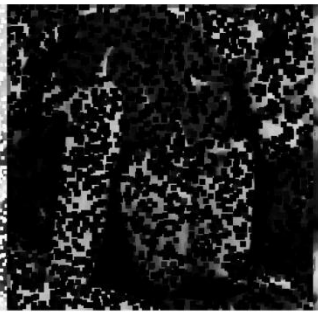
N-Rank 1 - 3x3

N-Rank 4 - 3x3

N-Rank 9 - 3x3

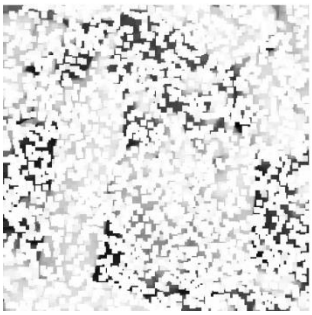


N-Rank 1 - 5x5



N-Rank 12 - 5x5

N-Rank 25 - 5x5



Problem 4:

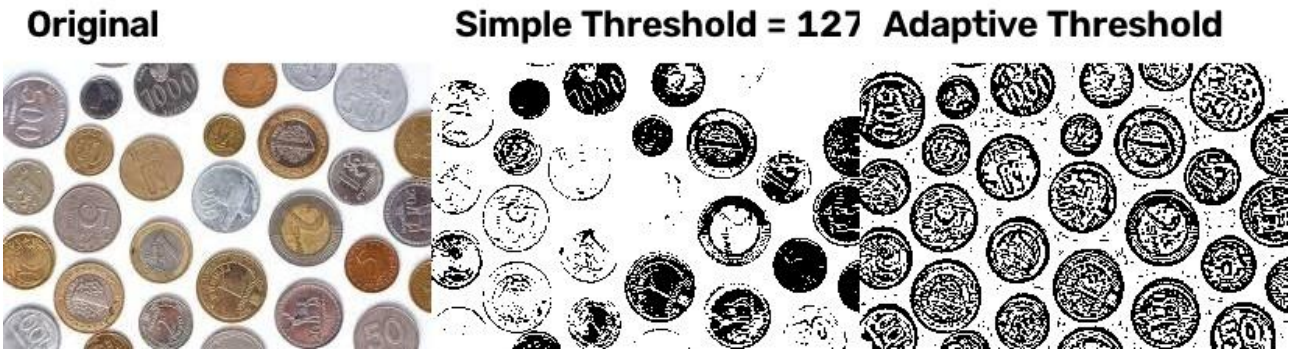
Process:

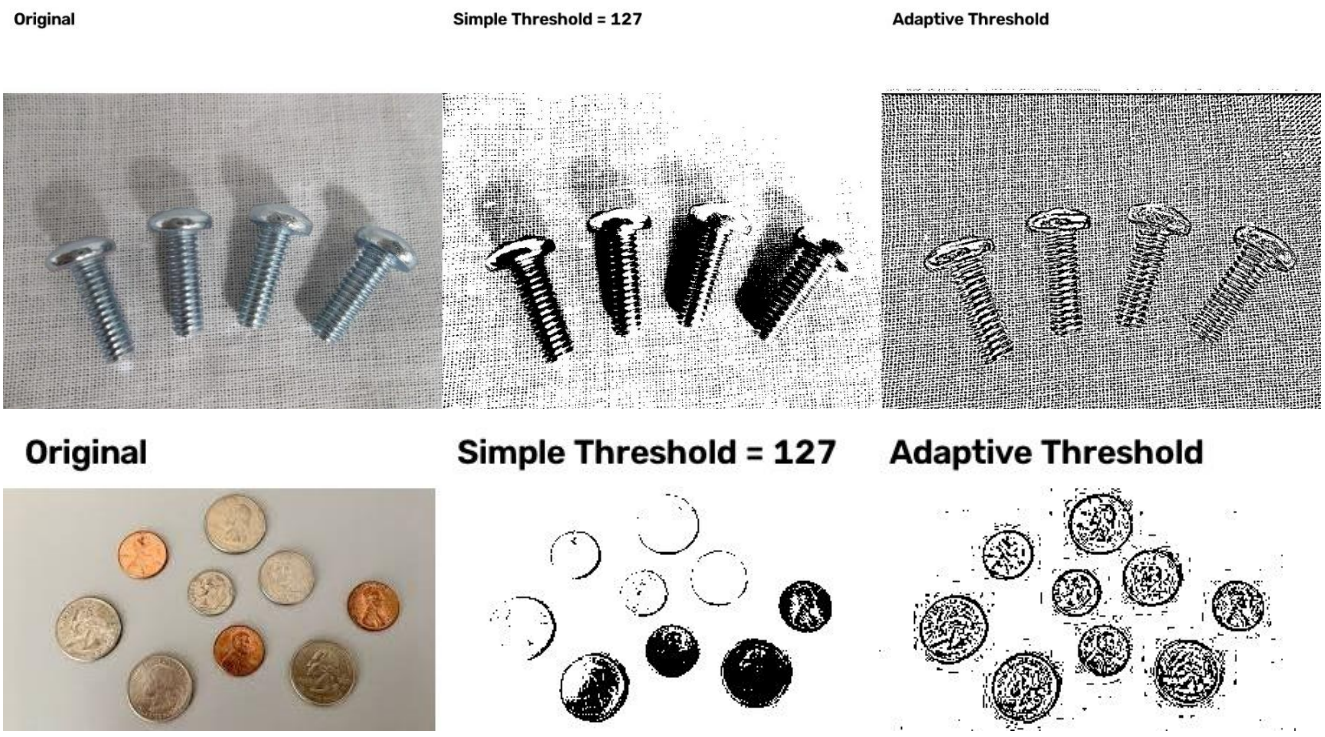
"4_Thresholding.py" was generated by ChatGPT using the prompt:

> "Write me a simple program using OpenCV and python to turn source images into greyscale, then run adaptive and simple thresholding on them. Finally, display the original image next to each kind of thresholding in a 1x3 orientation and save as a single image."

Before any thresholding can be run on an object, it is necessary to convert it into greyscale. This is because thresholding converts pixels on either side of a selected value into black or white pixels to create a binary image. It is impossible to determine how close to black or white red is. Simple thresholding uses a single value as the watershed. Adaptive thresholding operates in zones to preserve more detail. A value of 127 was set for the simple thresholding, as it falls in between 0 and 255.

Results:





There is a consistent trend between the simple and adaptive thresholding. Simple thresholding creates the clearest difference between objects and the background, while adaptive thresholding preserves the most detail and eliminates shadows, but introduces or fails to eliminate noise from the background.

Problem 5:

Process:

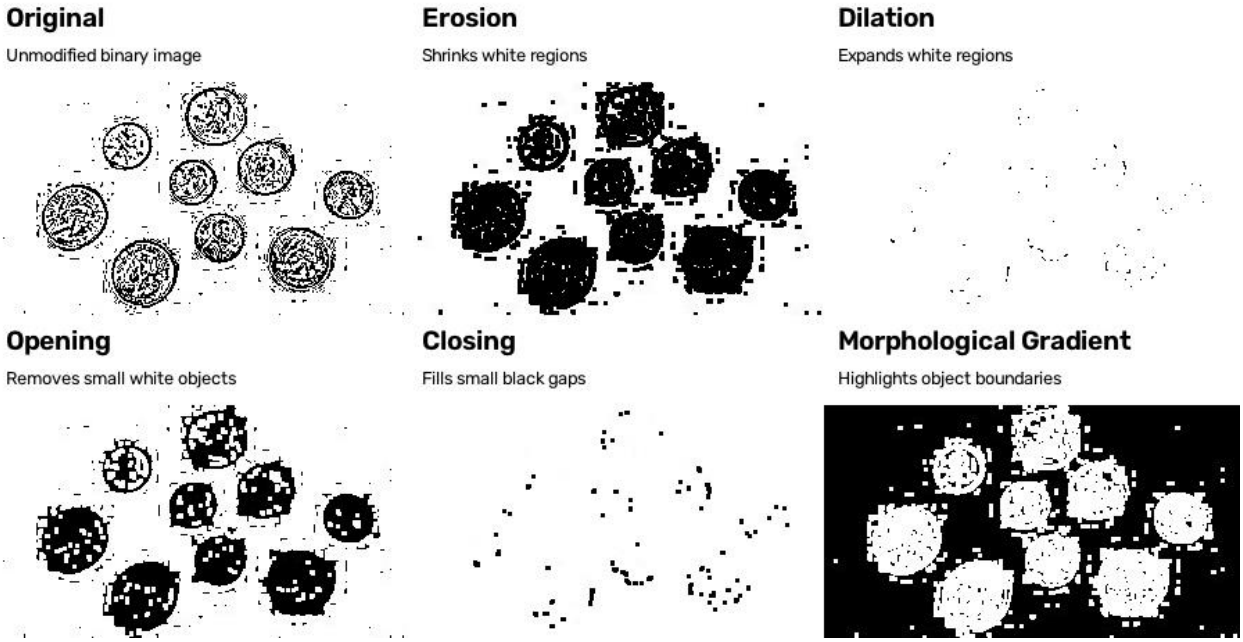
The code in "5_Morphology.py" was generated by ChatGPT with the following prompt:

> "Write me a simple program using OpenCV and python to take source images and run them through erosion, dilation, opening, closing, and morphological gradient. Display the original image next to each kind of transformation in a 2x3 orientation and save as a single image with labels describing the effect. The images are already binary, so don't waste time converting them to grayscale."

Results:

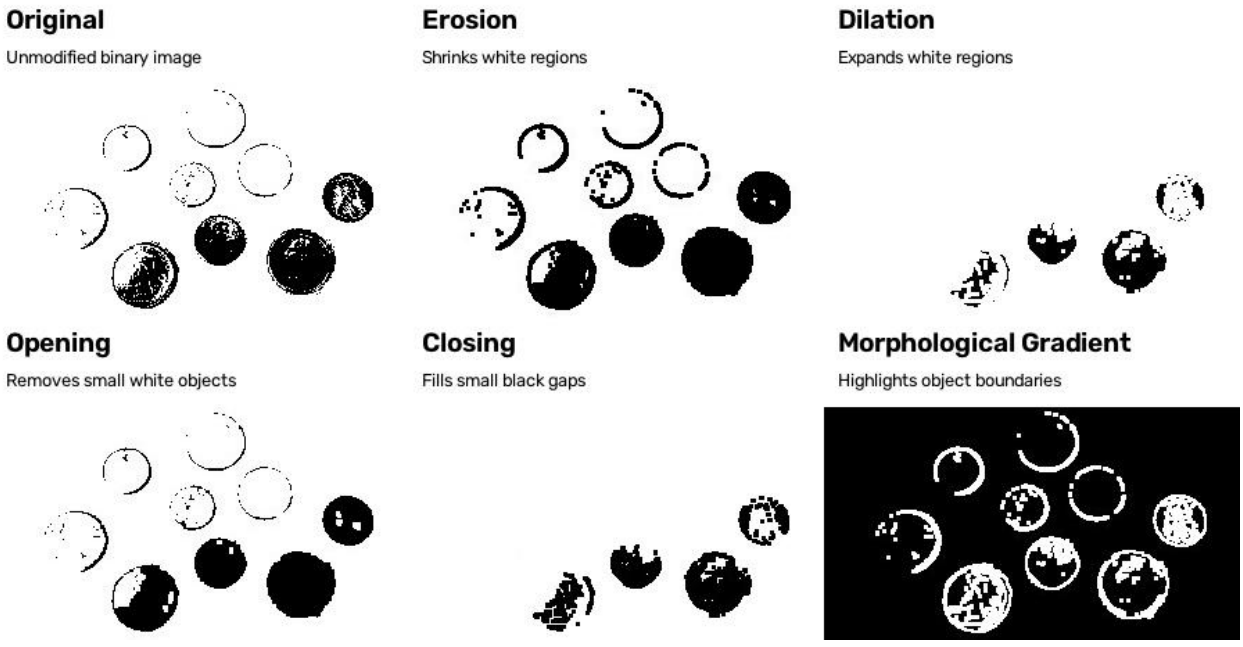
Coins 1, Adaptive

The operations on this image were all detrimental to the image quality. Erosion and the gradient highlight the difference between where and object is and where the background is.



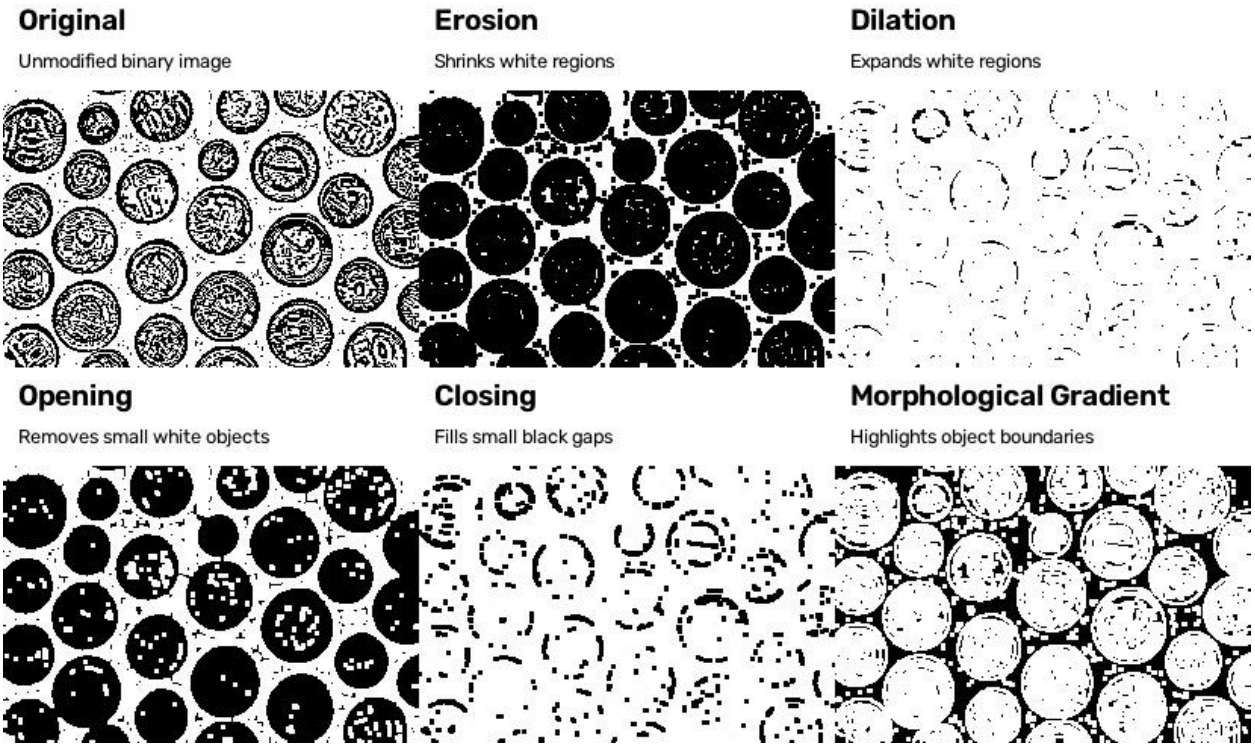
Coins 1, simple

A similar effect to the adaptive threshold is seen here, where details are removed. This image starts with much less noise, and erosion, opening, and gradient produce more clearly circular objects than the original.



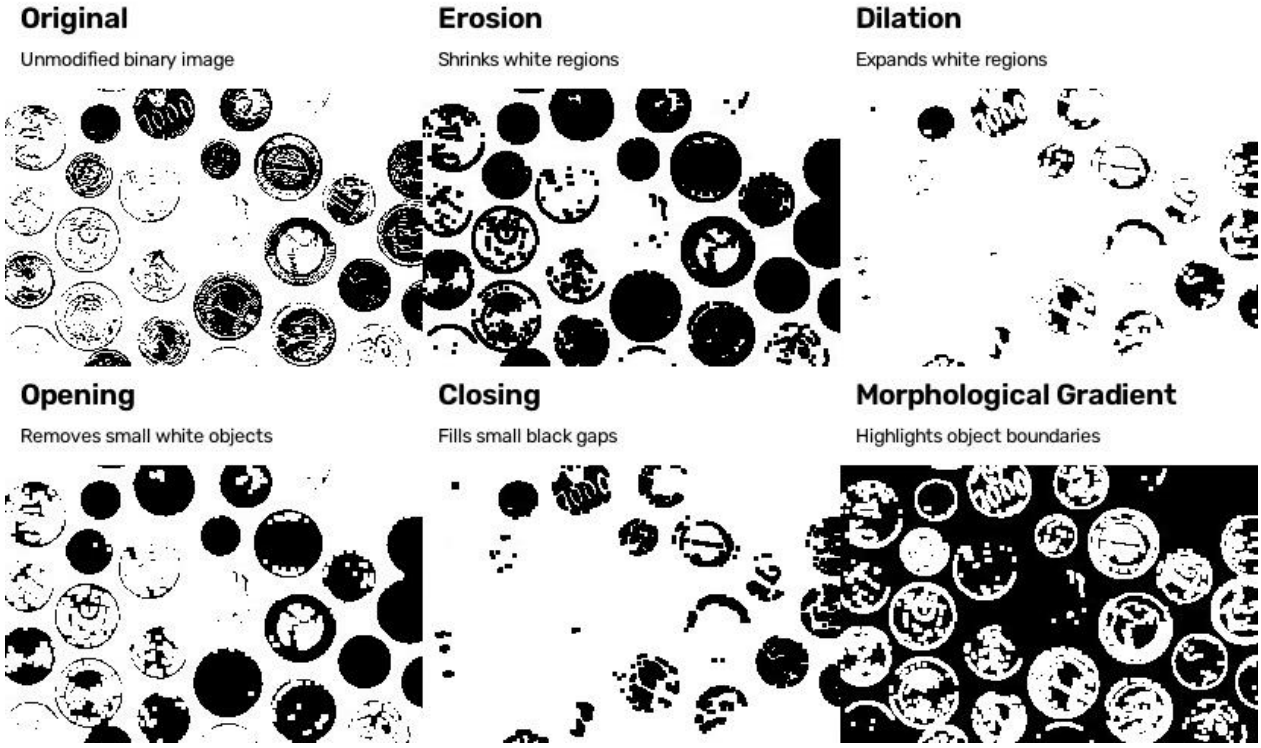
Coins 2, adaptive

The operations remove detail, but erosion, opening, and gradient do a good job of simplifying the images.



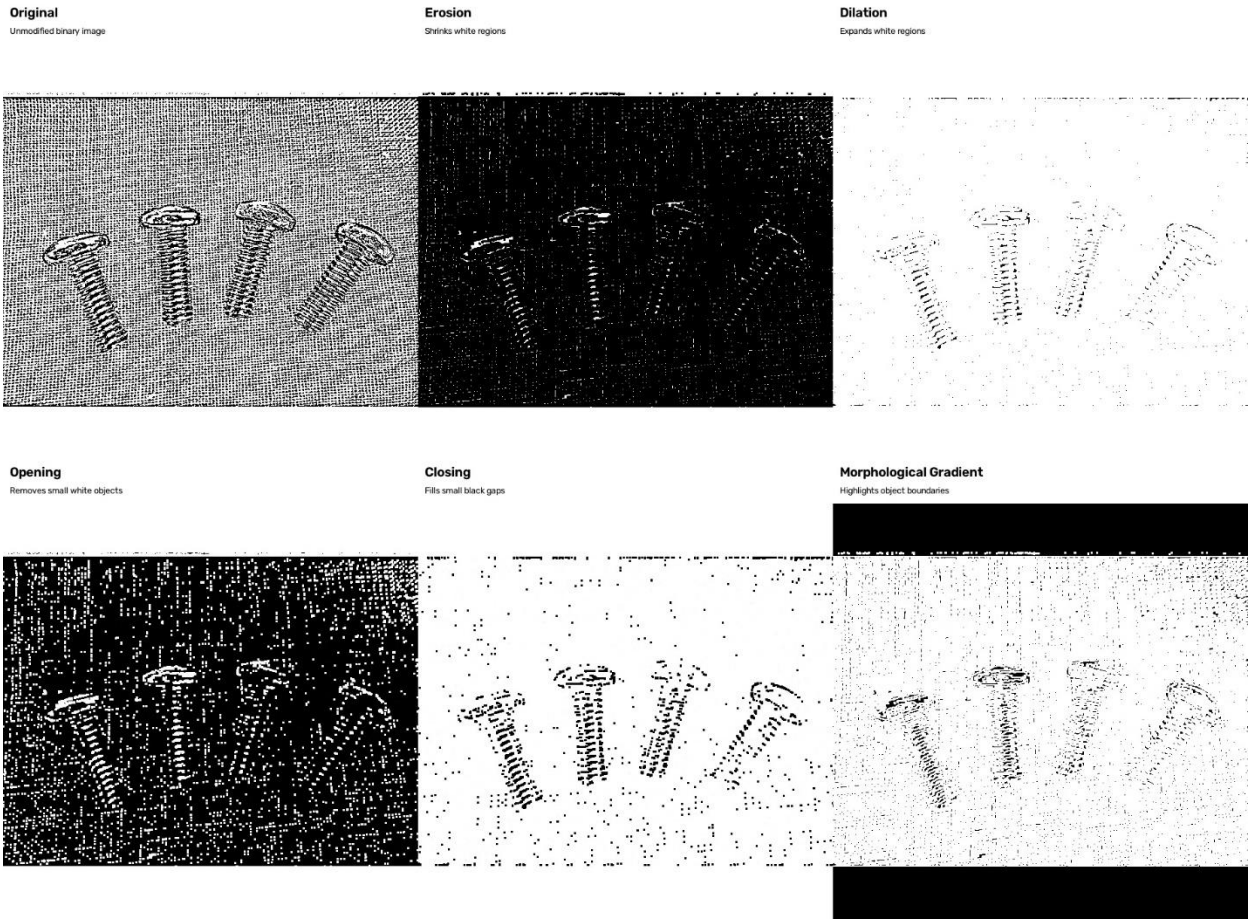
Coins 2, simple

Same as previous, detail is removed, but erosion, opening, and the gradient produce the most useful images for discerning individual objects.



Screws, adaptive

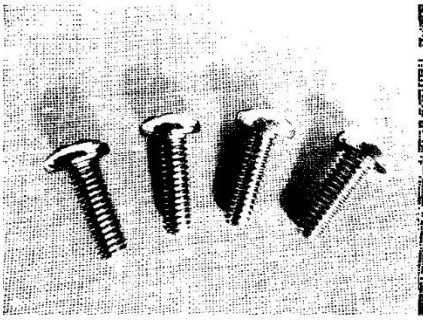
Dilation does the best job of removing the background by expanding the white regions in the speckle pattern. Adding an erosion operation afterwards results in the “closing image”, which restores much of the object lost to dilation with a small increase in noise.



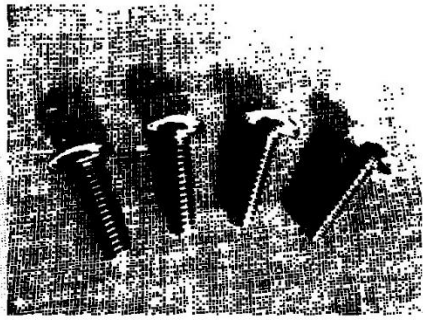
Screws, simple

Dilation does an excellent job of removing the background in this set. Unlike the previous, closing does not improve the image, but does still increase background noise.

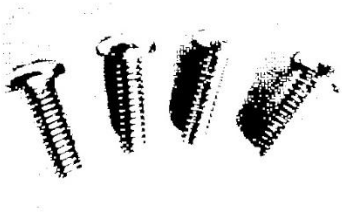
Original
Unmodified binary image



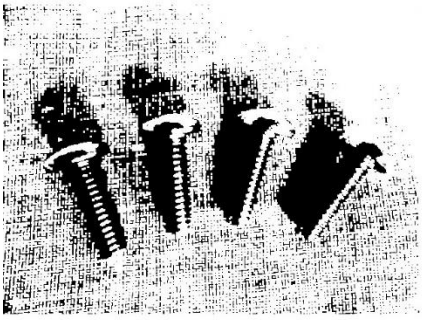
Erosion
Shrinks white regions



Dilation
Expands white regions



Opening
Removes small white objects



Closing
Fills small black gaps

Morphological Gradient
Highlights object boundaries